Lecture 15: Python Modules (Chapter 7)

유준수 (Joon Soo Yoo, Ph.D.)

2026년 4월 7일

방산첨단보안연구실 (DAS Lab)
전북대학교 첨단방위산업학과

7.4 The help function

모듈 탐색기: 도움말과 문서화 읽기

웹사이트의 색상: #RRGGBB

파이썬 모듈을 배우기 전, 웹 디자인에서 사용하는 색상 코드(HTML Color)를 이해해 봅시다.

- **16진수 포맷:** `#RRGGBB` (Red, Green, Blue)
- **값의 범위:** 0 ~ 255 (16진수로는 `00` ~ `FF`)
- **밝기 조절:** 값이 높을수록 밝은 색상이 됩니다.

색상 코드 예시

코드	색상명	설명
`#FF0000`	**Red**	빨강 가득, 나머지는 없음
`#000080`	**Navy**	파랑이 절반 정도인 어두운 파랑
`#FFFFFF`	**White**	모든 빛이 가득 (가장 밝음)

실전 예시: colors.py 모듈 설계

그래픽 디자이너를 위해 자주 쓰는 색상과 기능을 모은 `colors.py`를 만들어 봅시다.

colors.py 구성 요소

```
"""색상 이름과 16진수/RGB 값을 처리하는 함수 모음"""
RED = "#FF0000"   WHITE = "#FFFFFF"   # 색상 상수 정의
def _tohex(value):   # 내부용 함수 (언더바 시작)
    """정수를 16진수 문자열로 변환"""
def lighten(color):   # 외부용 함수
    """색상의 밝기를 32만큼 증가시킴"""
```

비밀의 언더바 (_)

함수 이름이 **언더바(_)**로 시작하면, "이 함수는 모듈 내부에서만 쓸 테니 밖에서는 부르지 마세요"라는 약속입니다.

help() 함수: 모듈 설명서 자동 생성

파이썬의 `help()` 함수를 쓰면, 우리가 작성한 **Docstring**을 모아 멋진 설명서를 보여줍니다.

도움말 호출하기

```
import colors

help(colors)
```

- **NAME:** 모듈의 이름과 한 줄 요약
- **FUNCTIONS:** 사용 가능한 함수 목록 (알파벳 순서 정렬)
- **DATA:** 정의된 상수(RED, BLUE 등)의 값
- **특징:** `_tohex`처럼 언더바로 시작하는 내부 함수는 숨겨집니다.

Task: 모듈의 세계 탐험하기

Mission: 파이썬이 기본으로 제공하는 모듈의 설명서를 읽어보세요.

1. 터미널이나 파이썬 쉘을 켭니다.
2. `import random`, `import statistics`를 입력합니다.
3. `help(random)`을 입력하고 화살표 키로 내용을 훑어보세요.
4. **팁:** 구체적인 함수가 궁금할 땐 `help(random.randint)` 처럼 마침표를 찍어 물어보세요!
5. 다 읽었다면 **'q'** 키를 눌러 빠져나옵니다.

7.5 Finding modules

표준 라이브러리 vs PyPI: 나에게 필요한 도구 찾기

표준 라이브러리: "Batteries Included"

파이썬은 설치와 동시에 바로 사용할 수 있는 200개 이상의 강력한 모듈을 제공합니다.

- **철학:** "건전지 포함(Batteries Included)" - 별도 설치 없이 대부분의 프로그래밍 작업이 가능함.
- **공식 문서:** `python.org`의 Library Reference에서 카테고리별로 확인 가능합니다.

주요 표준 라이브러리 예시

모듈명	주요 기능
`math`	수학적 함수 및 상수 (`sqrt`, `pi` 등)
`os`	운영체제(파일 시스템, 환경 변수 등)와 상호작용
`random`	의사 난수(Pseudo-random) 생성
`datetime`	날짜 및 시간 데이터 처리

PyPI: 전 세계 개발자들의 공유 창고

PyPI(Python Package Index)는 공식적인 제3자(Third-party) 소프트웨어 라이브러리입니다.

- **오픈 소스:** 누구나 모듈을 개발하여 공유할 수 있으며, 대부분 무료입니다.
- **특화된 기능:** 표준 라이브러리보다 더 강력하거나 편리한 기능을 제공합니다.

주요 PyPI 모듈

모듈명	용도
`numpy`	수치 계산 및 행렬 연산의 필수 패키지
`pandas`	데이터 분석 및 시계열 데이터 처리
`tensorflow`	딥러닝 및 머신러닝 플랫폼
`matplotlib`	데이터 시각화 및 그래프 생성

표준 라이브러리 활용: datetime

파이썬 내장 모듈인 datetime은 추가 설치 없이 바로 쓸 수 있지만, 상세한 계산은 직접 구현해야 합니다.

datetime을 이용한 나이 계산

```
from datetime import date


birth = date(2005, 3, 14)
today = date(2020, 3, 14)   # 15년 후의 오늘
delta = today - birth   # 두 날짜의 차이(timedelta) 구하기


years = delta.days // 365   # 일수를 365로 나눈 몫
days = delta.days % 365   # 남은 일수(나머지)
print(f"You are {years} years and {days} days old.")
```

제3자 모듈 활용: arrow

PyPI에서 제공하는 arrow 모듈을 사용하면 복잡한 날짜 연산과 출력을 훨씬 직관적으로 처리할 수 있습니다.

arrow를 이용한 인간 중심적(Humanize) 출력

```
import arrow   # pip install arrow 로 설치 필요
birth = arrow.get(2005, 3, 14)
 # "15 years ago" 처럼 사람이 읽기 쉽게 변환하여 출력
print(birth.humanize())
```

가이드라인

기본적인 날짜 기록은 표준 datetime을 쓰되, 사용자 인터페이스(UI)에 보여주거나 복잡한 시계열 분석이 필요할 때는 arrow, pandas 같은 검증된 외부 모듈을 활용하세요.

Task: 나의 생존 일수와 "인간적인" 나이 출력

Mission: datetime과 arrow를 모두 사용하여 나이를 계산해 보세요.

1. `datetime.date`를 사용하여 자신이 태어난 지 몇 일째인지 계산합니다.
2. `arrow` 모듈을 사용하여 자신의 나이를 "18 years ago"와 같이 사람이 읽기 편한(Humanize) 방식으로 출력하세요.
3. **도전:** `arrow`의 설명서를 참고하여 "18 years 4 months and 7 days old" 처럼 상세히 출력해 보세요.

9.1 Modifying and Iterating Lists

리스트의 변화와 탐색: 데이터 추가, 삭제, 반복

리스트 수정하기: 추가와 삭제

리스트는 한 번 만들고 끝나는 것이 아니라, 필요에 따라 요소를 추가하거나 삭제할 수 있습니다.

- `append(x)`: 리스트의 **맨 마지막**에 요소 `x`를 추가합니다.
- `remove(x)`: 리스트에서 특정 요소 `x`를 찾아 삭제합니다.
- `pop()`: 리스트의 **맨 마지막** 요소를 꺼내어 삭제합니다.

실전 예시: 연구원 명단 관리

```
lab_members = ["Yongjun", "Jinsung", "Minseok"]
lab_members.append("Moud")   # 새 멤버 추가
lab_members.remove("Yongjun")   # 특정 멤버 제외
lab_members.pop()   # 마지막에 들어온 멤버(Moud) 제외

print(lab_members)   # 결과: ['Jinsung', 'Minseok']
```

리스트 탐색 (1): 직접 반복 (Element-based)

리스트에 있는 데이터를 하나씩 순서대로 바로 꺼내서 사용하는 가장 직관적인 방법입니다.

직접 반복 예시: 연구원 명단 출력

```
lab_members = ["Yongjun", "Jinsung", "Minseok"]


for member in lab_members:
    print(f"안녕하세요, {member} 연구원님!")
```

- **작동 원리:** 반복문이 돌 때마다 `member` 변수에 리스트의 요소가 하나씩 자동으로 담깁니다.
- **장점:** 코드가 읽기 쉽고 간결합니다 (Pythonic 방식).

리스트 탐색 (2): 인덱스 반복 (Index-based)

데이터의 값뿐만 아니라 **'몇 번째(번호)'** 인지가 중요할 때는 인덱스를 활용합니다.

인덱스 반복 예시: 순번과 함께 출력

```
lab_members = ["Yongjun", "Jinsung", "Minseok"]


# len()으로 개수를 구하고 range()로 번호를 생성합니다.
for i in range(len(lab_members)):
    member = lab_members[i]
    print(f"제 {i+1}연구원: {member}")
```

- `len()`: 리스트의 전체 길이를 알려주는 함수입니다.
- `range(n)`: 0부터 $n-1$까지의 숫자 리스트를 만들어줍니다.
- **장점:** 데이터의 위치(Index)를 자유롭게 조절하거나 활용할 수 있습니다.

Task: 스포츠 목록과 검색 시스템

Mission 1: 스포츠 리스트 수정하기

1. `sports = ["baseball", "football", "tennis", "table tennis"]` 리스트를 만드세요.
2. `volleyball`을 추가(`append`)하세요.
3. `football`을 삭제(`remove`)하고 `soccer`를 추가하세요.

Mission 2: 검색 시스템 (Simple Search)

1. 반복문을 사용하여 리스트 안에 "soccer"가 있는지 확인하세요.
2. 만약 있다면 "Found!"를 출력하는 프로그램을 작성하세요.

Any Questions?